

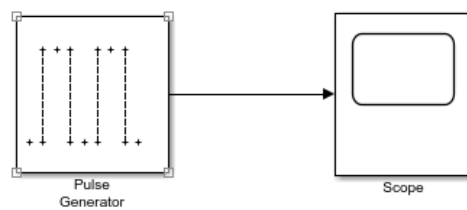
TP1 - R305

Prise en main

On commence par ouvrir Matlab et on clique sur Simulink.

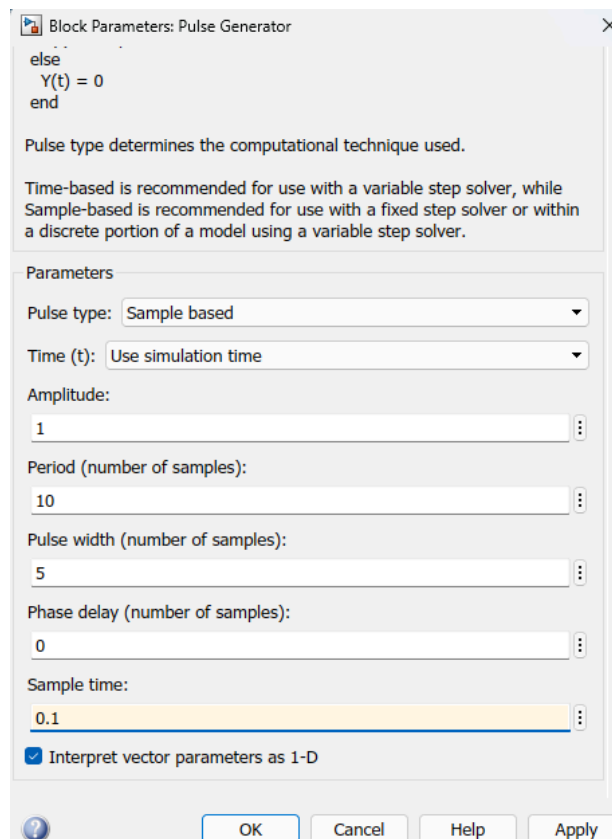
On ajoute ensuite un Pulse Generator et un Scope :

- Simulink > Sources > Pulse generator
- Simulink > Sinks > Scope

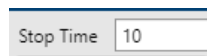


On règle le Pulse Generator :

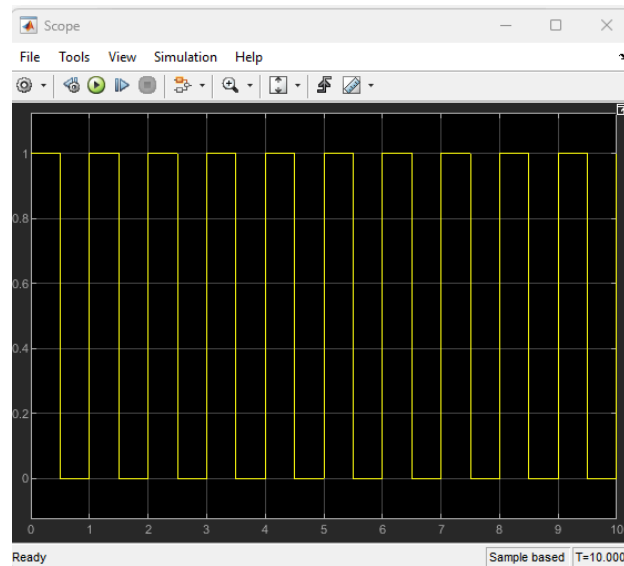
Pulse type sur la valeur Sample based et le paramètre Sample Time à 0.1 seconde.



On règle le stop time sur 10 :



On obtiens ceci :



1. Quelle est la durée de la simulation ?

La durée de la simulation est de 10 secondes.

2. Quelle est la durée du signal carré à l'état haut ?

La durée à l'état haut est de 0.5 secondes.

3. Quelle est la durée du signal carré à l'état bas ?

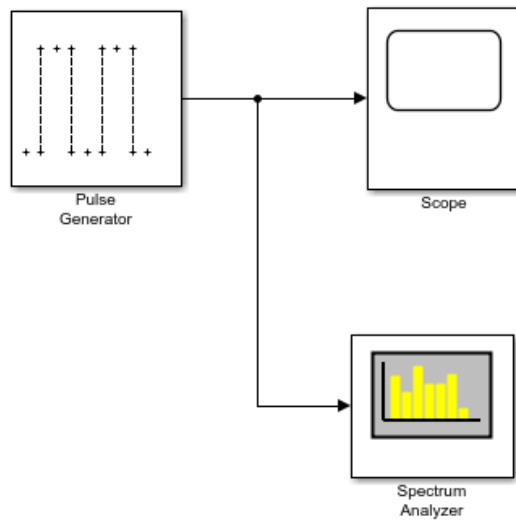
La durée à l'état bas est de 0.5 secondes.

4. Quelle est la période du signal carré ?

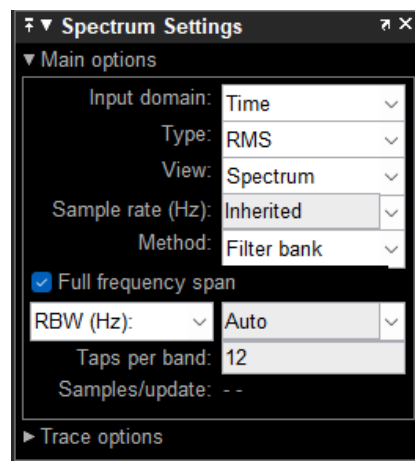
La période est de 1 seconde.

On ajoute un Spectrum Analyser qu'on relie au Pulse Generator :

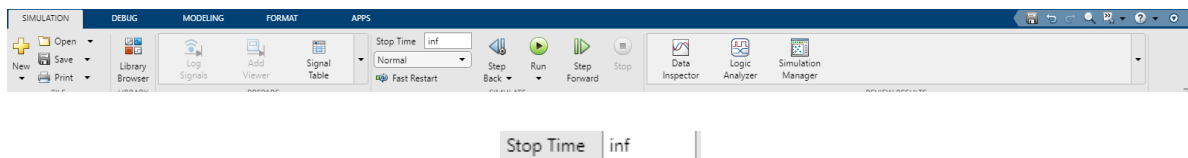
DSP System Toolbox > Sinks > Spectrum analyser



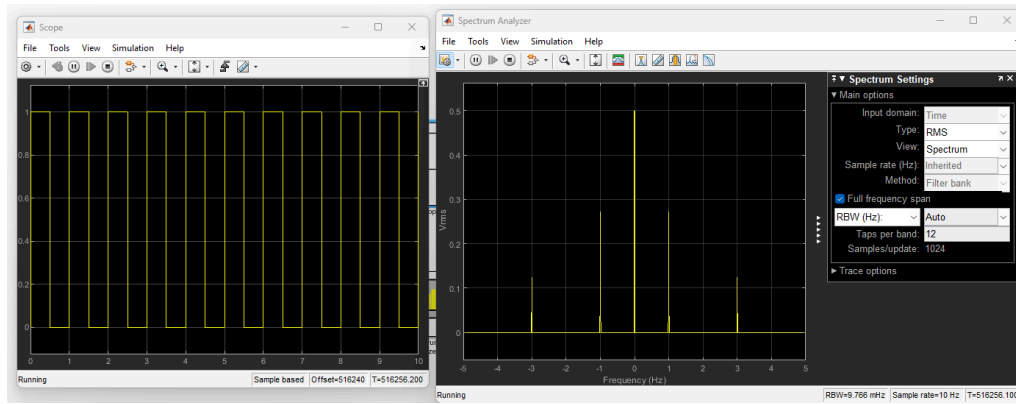
On règle le Spectrum Analyser dans le menu **View > Spectrum Setting** :
mettre le paramètre **Type** sur **RMS** .



On met le stop time sur inf :



on obtiens ceci :



1. Relevez la fréquence maximale de l'axe des fréquences. Que représente cette valeur ?

La fréquence maximale de l'axe des fréquences est 5Hz, elle correspond à la moitié de la fréquence d'échantillonnage conformément au théorème de Nyquist-Shannon.

2. Est-ce en accord avec la période d'échantillonnage du **Pulse generator** ?

Oui car la fréquence max (5Hz) correspond la moitié du taux d'échantillonnage (10Hz).

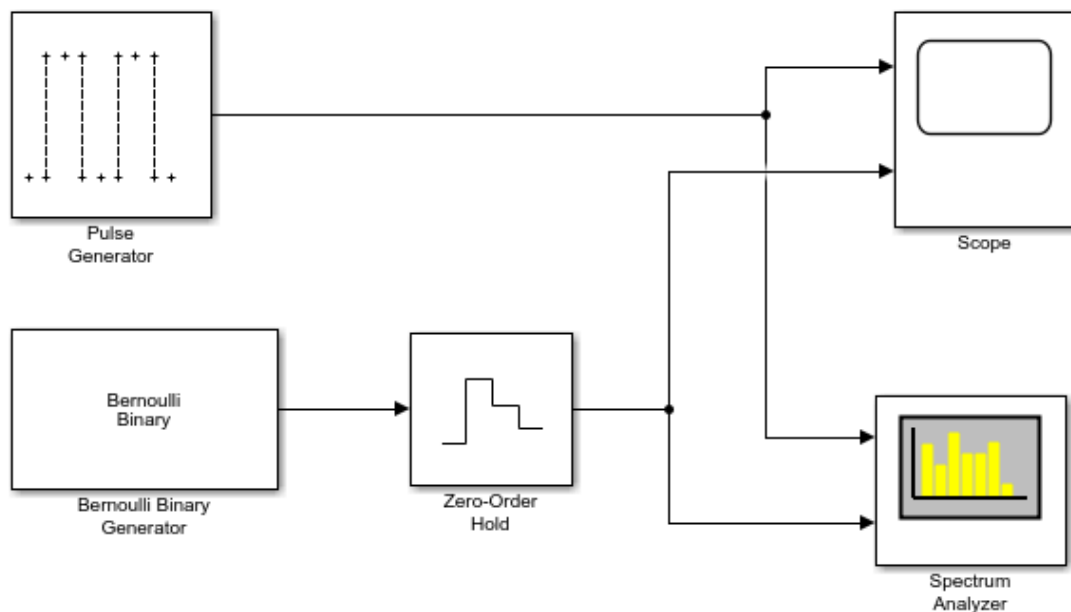
3. Que représente la raie à la fréquence de 0 Hz ?

Elle représente la composante continue du signal.

On ajoute ensuite la boîte Bernouilli Binary et la boîte Zero-Order Hold :

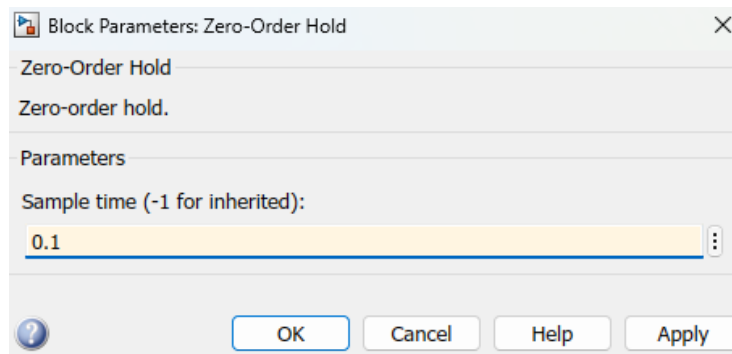
Communications Toolbox > Comm Sources > Random Data Sources > Bernouilli Binary

Simulink > Discrete > Zero-Order Hold

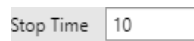


On modifie les paramètres du bloc Zero-Order Hold :

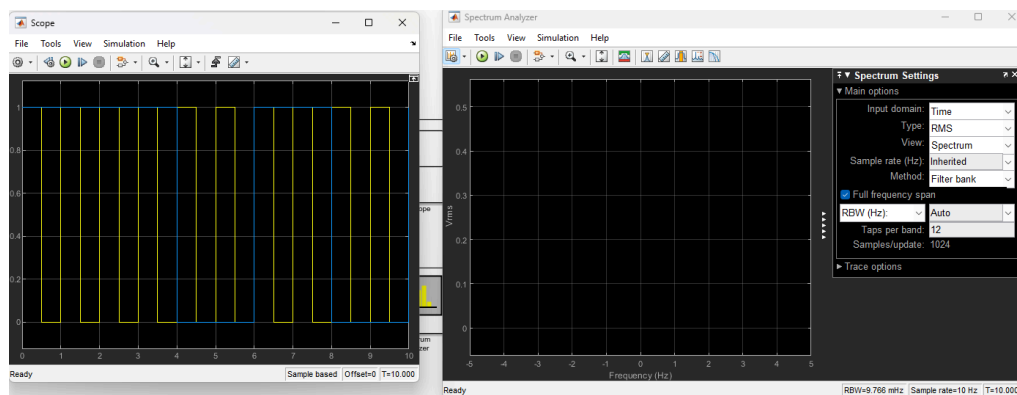
Sample time à 0.1



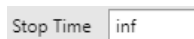
On fixe le stop time a 10 sec :



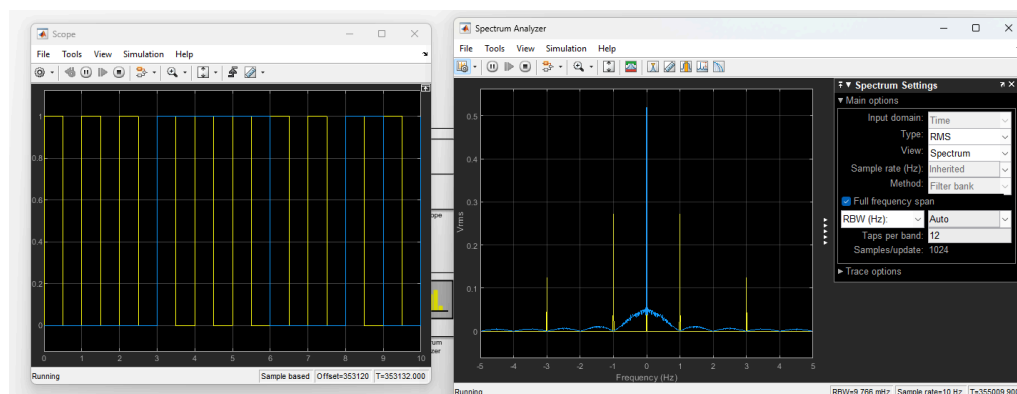
On obtiens ceci :



On fixe le stop time a inf :



On obtiens ceci :



1. Quelle est la différence entre les deux spectres ?

La différence entre les deux spectres est que le spectre jaune est discontinu et le spectre bleu est continu.

2. Relever l'amplitude et la fréquence des différentes raies et comparer les valeurs avec la série de Fourier d'un signal carré :

$$S_c(t) = \frac{4}{\pi} \sum_{n=0}^{+\infty} \frac{\sin(2n+1)2\pi ft}{2n+1}$$

Les amplitudes de notre signal carré sont 0.5 en 0Hz, 0.27 en 1Hz et 0.12 en 3Hz et pour la série de Fourier les amplitudes sont : 1.27 en 0Hz, 0.63 en 1Hz et 0.42 en 3Hz. (PAS DU TOUT COMPRIS POURQUOI)

3. Relever les fréquences pour lesquelles le spectre du signal aléatoire s'annule. Donner alors la bande passante du signal.

Les fréquences pour lesquelles le spectre s'annule sont -5, -4, -3, -2, -1, 1, 2, 3, 4, 5.

La bande passante va de -1 à 1 donc elle est de 2Hz.

4. Quelle est la particularité du signal généré aléatoirement ?

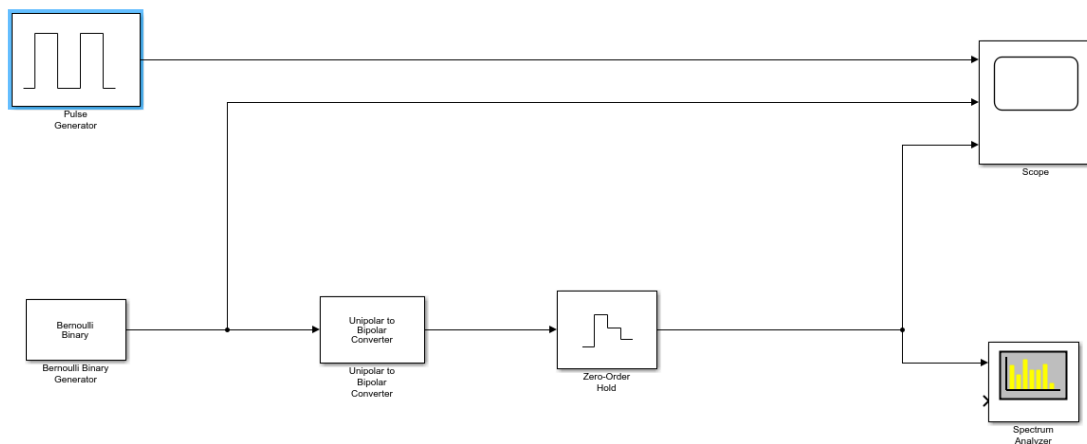
Il est non périodique.

Les codes en lignes

Codes NRZ

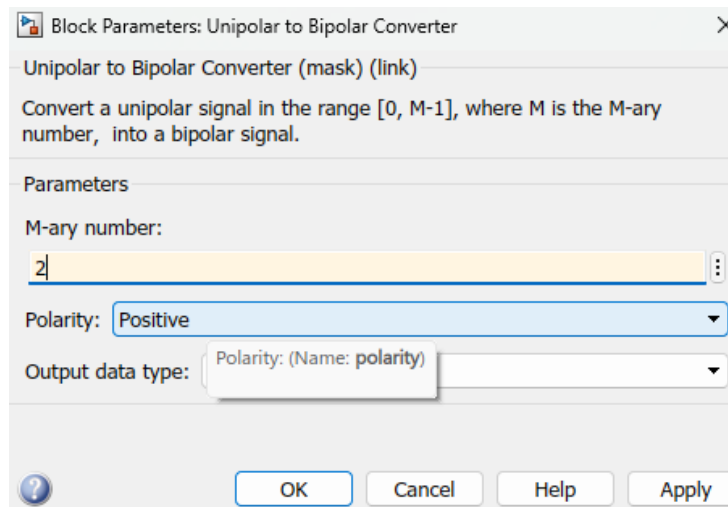
On met en place ce schéma :

Communications Toolbox > Utility Blocs > Unipolar to bipolar Converter



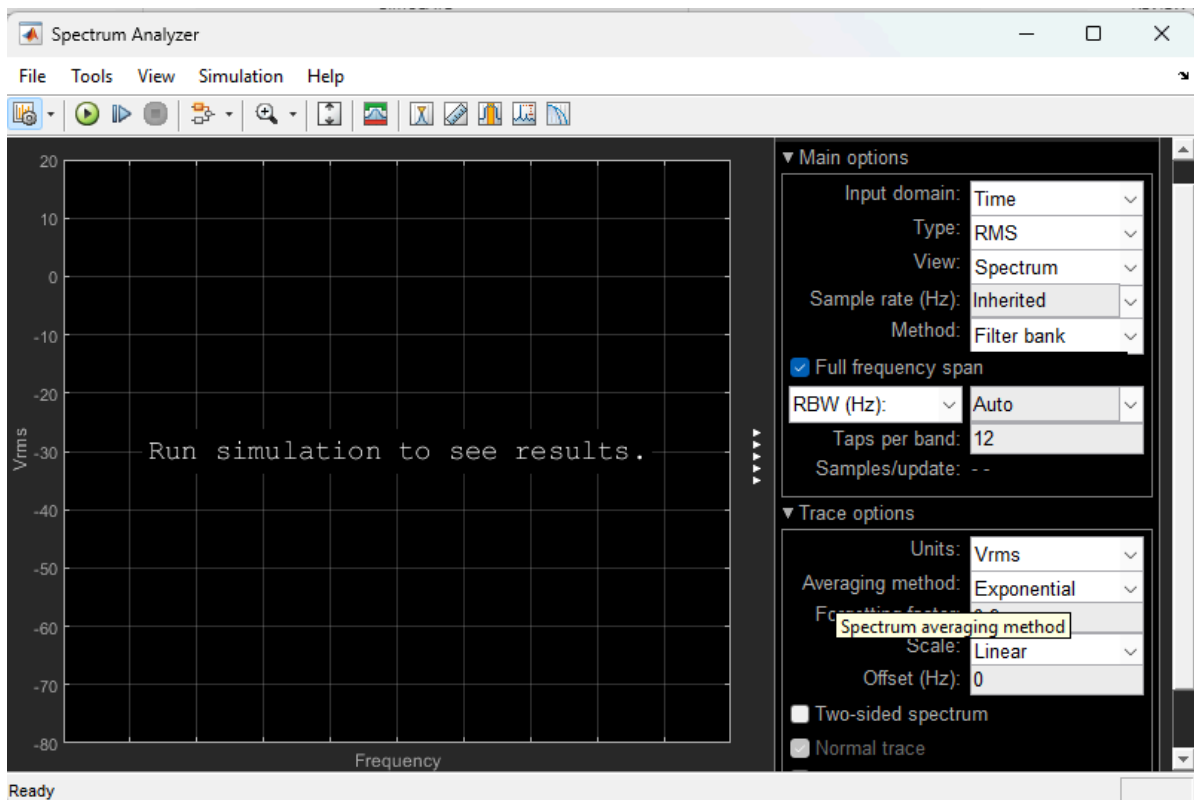
On règle le bloc Unipolar to Bipolar Converter :

M-ary number a 2.

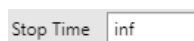


On règle le Spectrum analyzer :

Trace options décochez la case Two-Sided spectrum

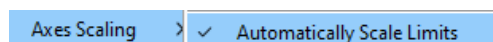


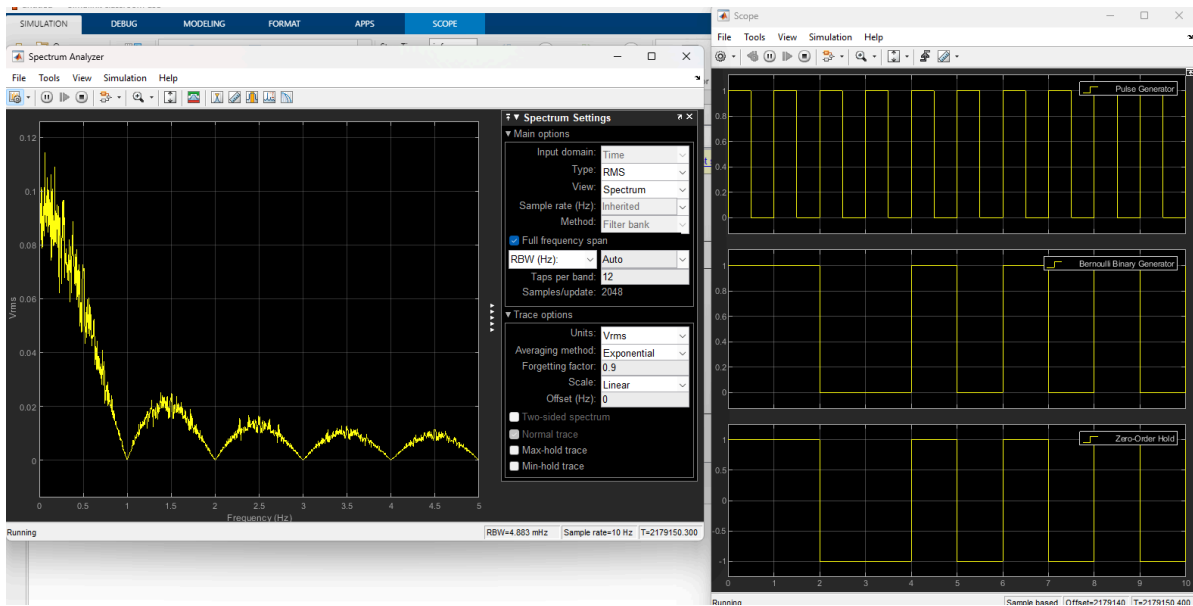
On règle le stop time a inf :



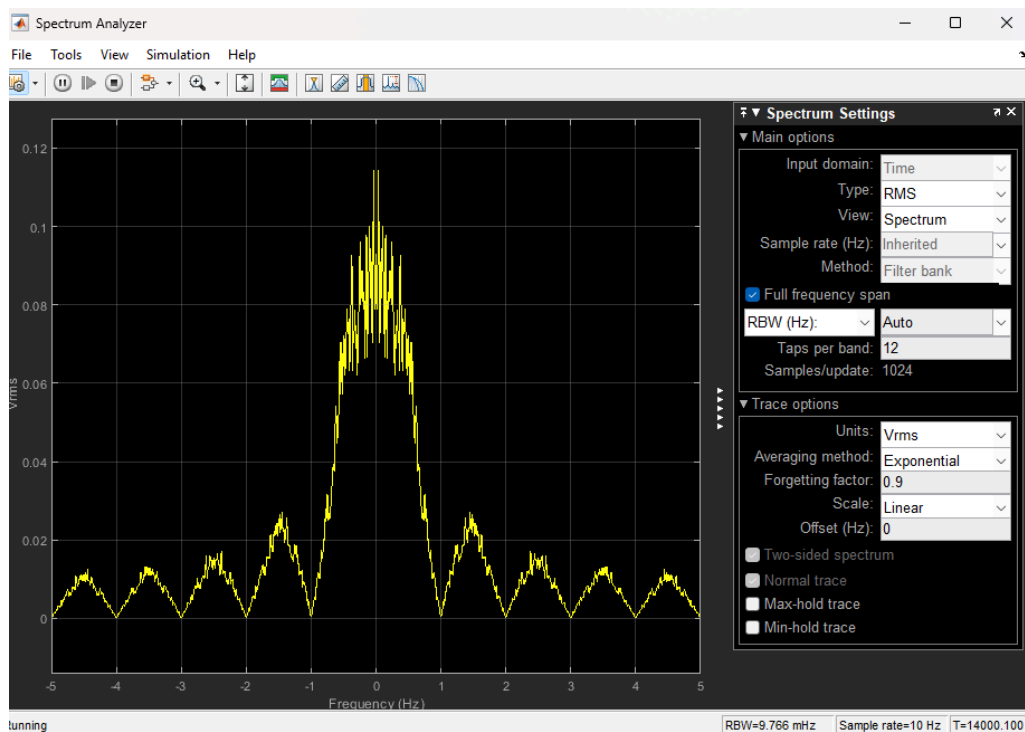
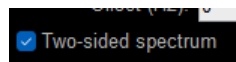
On obtiens ceci :

Pour régler l'axe Y il faut aller dans tools Axes Scaling automatically scale limits





Pour mesurer la bande passante, il faut réactiver le two-sided spectrum dans le spectrum analyser :



1. Mesurer la bande passante du signal NRZ.

On voit bien que la bande va de -1 à 1 donc la bande passante est de 2 Hz.

2. Quel est le défaut majeur du signal NRZ ?

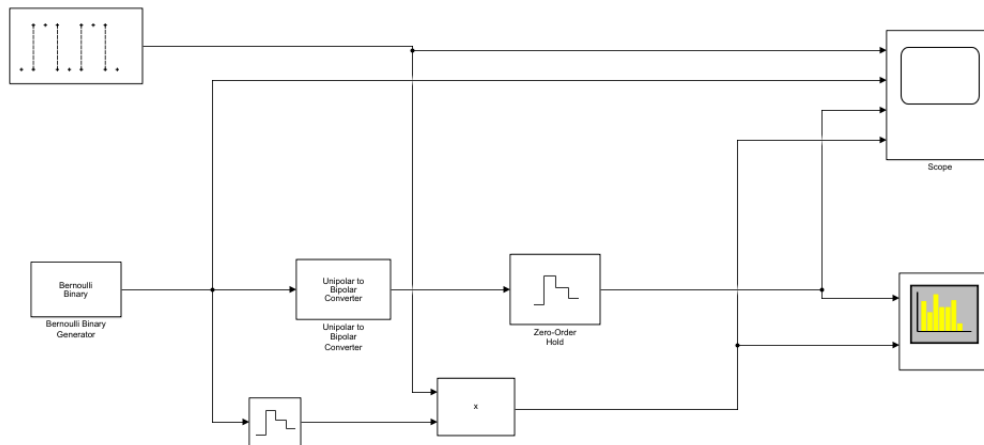
En cas de longues suites de bits identiques, il n'y a **aucune transition**, donc **perte de synchronisation** possible entre émetteur et récepteur.

Code RZ unipolaire

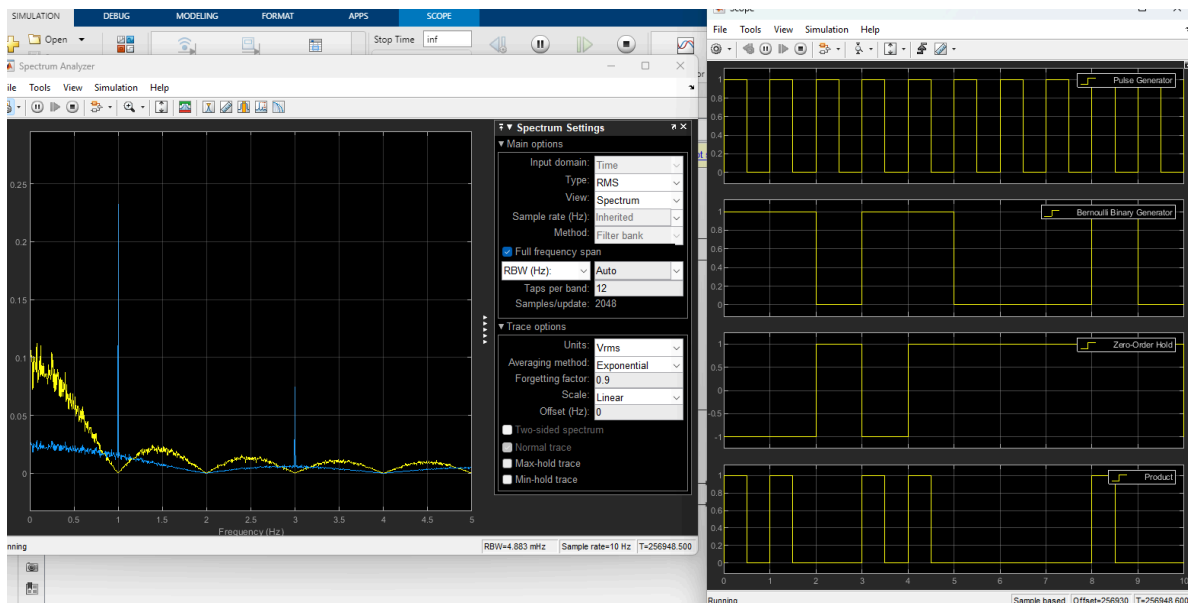
On ajoute un bloc Zero-Order Hold et un bloc Product :

Simulink > Math Operations > Product

On remet le sample time a 0.1 dans le nouveau Zero-Order Hold



On obtiens ceci :



1. Quelle est la particularité du spectre du signal RZ unipolaire ?

Il ne contient pas de composante continue

2. Relevez l'amplitude et la fréquence des deux raies.

Il y a une raie a 0.23 en 1Hz et une raie a 0.07 en 3Hz.

3. Quelle est la bande passante du signal RZ unipolaire ?

La bande passante va de -2 a 2 donc vaut 4 Hz.

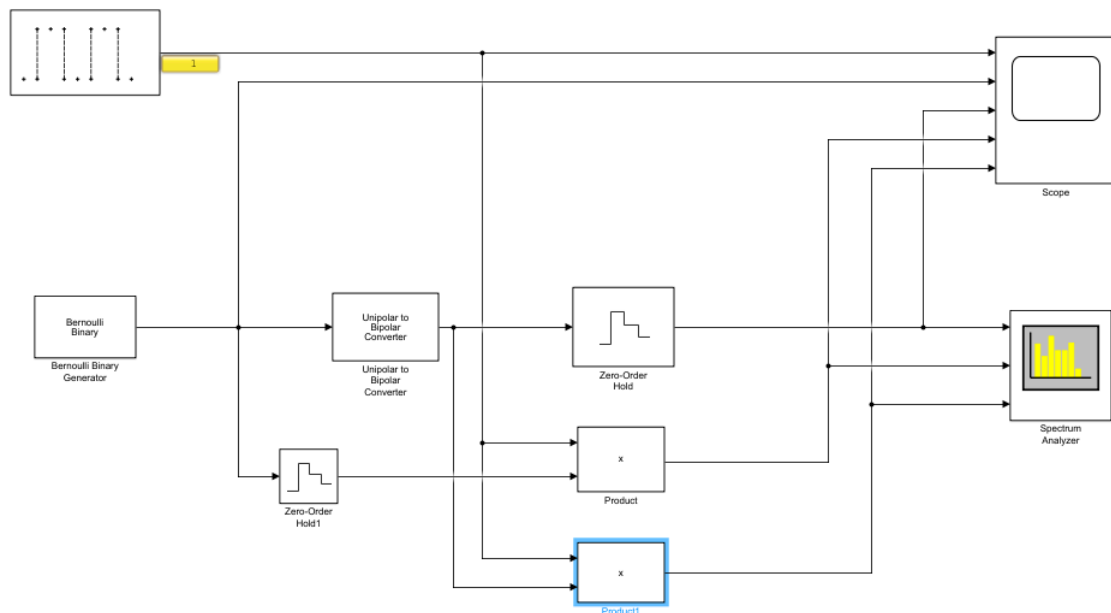
4. A quoi peuvent servir les deux raies du spectre ?

Elles peuvent servir à extraire ou synchroniser l'horloge du signal reçu.

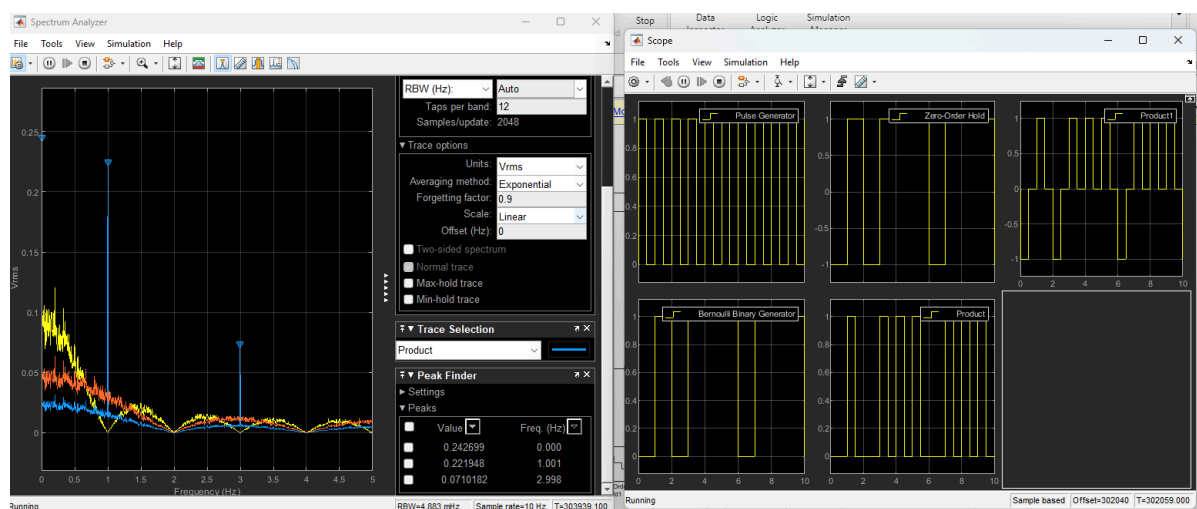
Code RZ bipolaire

On ajoute un bloc Product :

Simulink > Math Operations > Product



On obtiens :



1. Donner la bande passante du code RZ bipolaire.

La bande passante va de -2 à 2 donc elle vaut 4Hz

2. Quelle est l'amplitude du spectre à 0 Hz ?

L'amplitude à 0 Hz est d'environ 0.05

Le code Manchester

1. Rappeler la table de vérité d'un OU exclusif

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

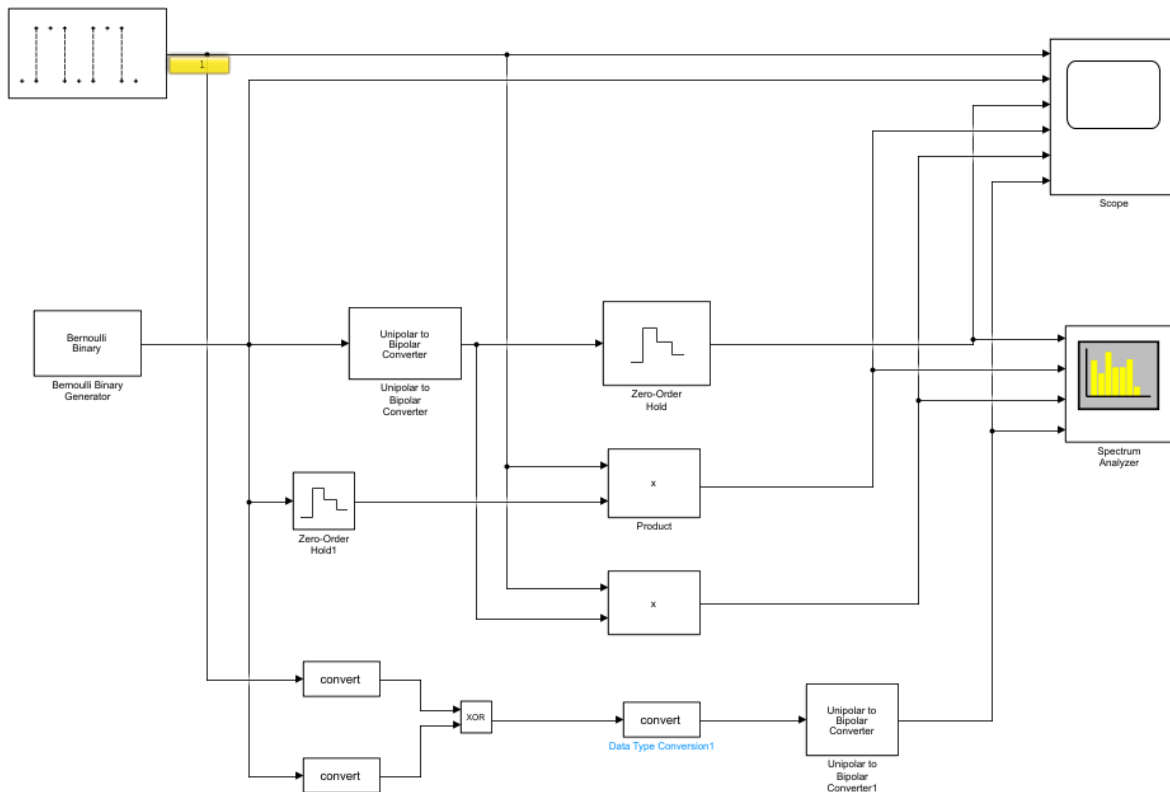
2. Tracer le chronogramme du code Manchester de la figure avec la suite binaire proposée.

On dit merci à Nicolas

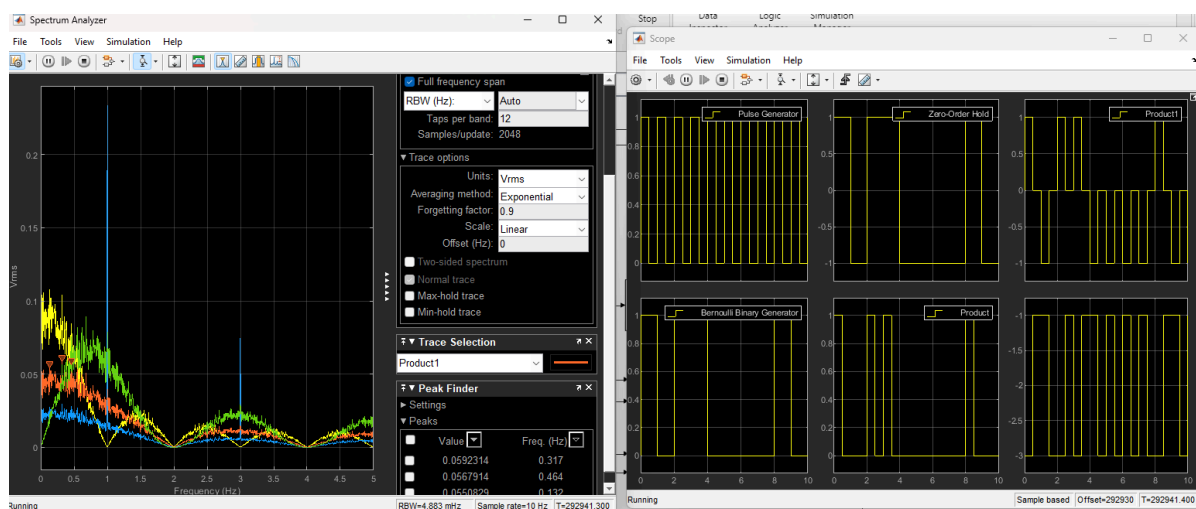
- Un **bit 1** → transition **haut** → **bas** au milieu du bit.
- Un **bit 0** → transition **bas** → **haut** au milieu du bit.

Bit (t →)	Data	Manchester 1 ^{re} moitié (H1)	Manchester 2 ^e moitié (H2)	Transition mi-bit
0-1 s	1	0	1	bas → haut
1-2 s	1	0	1	bas → haut
2-3 s	1	0	1	bas → haut
3-4 s	0	1	0	haut → bas
4-5 s	1	0	1	bas → haut
5-6 s	0	1	0	haut → bas
6-7 s	1	0	1	bas → haut
7-8 s	1	0	1	bas → haut
8-9 s	0	1	0	haut → bas
9-10 s	1	0	1	bas → haut

On insère dans notre schéma 3 boîtes Convert, une boîte logical operator en XOR et un Unipolar to Bipolar Converter :



On obtiens :



1. Vérifier que le code Manchester est correct en le comparant à la figure du travail préparatoire.

Oui, le signal présente bien une **transition à chaque milieu de bit**.

Les **1** donnent une transition **haut → bas** et les **0** une **bas → haut** → conforme à la figure préparatoire

2. Que pouvez-vous remarquer sur la DSP du code Manchester ?

Le spectre Manchester est **plus étalé** que NRZ et **centré autour de la fréquence $1/(2T)$** , sans raie à 0 Hz

3. Mesurer l'évolution spectrale du code Manchester.

Le spectre Manchester est **plus étalé** que NRZ et **centré autour de la fréquence $1/(2T)$** , sans raie à 0 Hz → **pas de composante continue**.

La **fréquence d'horloge** correspond au **nombre de bits envoyés par seconde**.

Tu as :

- Sample time = 0,1 s
- Period (number of samples) = 10
→ donc $T_b = 10 \times 0,1 = 1 \text{ s}$

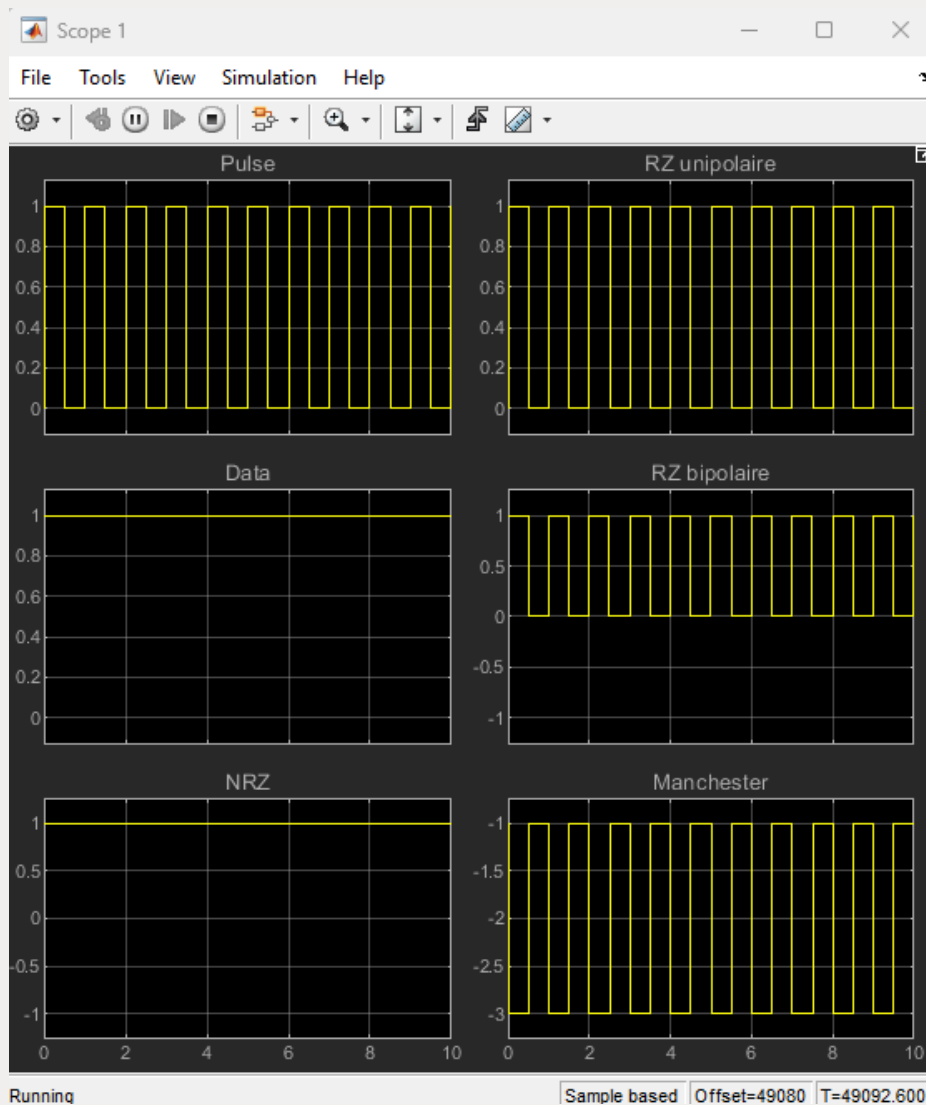
$$f_{horloge} = \frac{1}{1} = 1 \text{ Hz}$$

👉 **Fréquence d'horloge = 1 Hz, c'est-à-dire 1 bit transmis par seconde.**

Compléter le tableau en observant les DSP des différents codes.

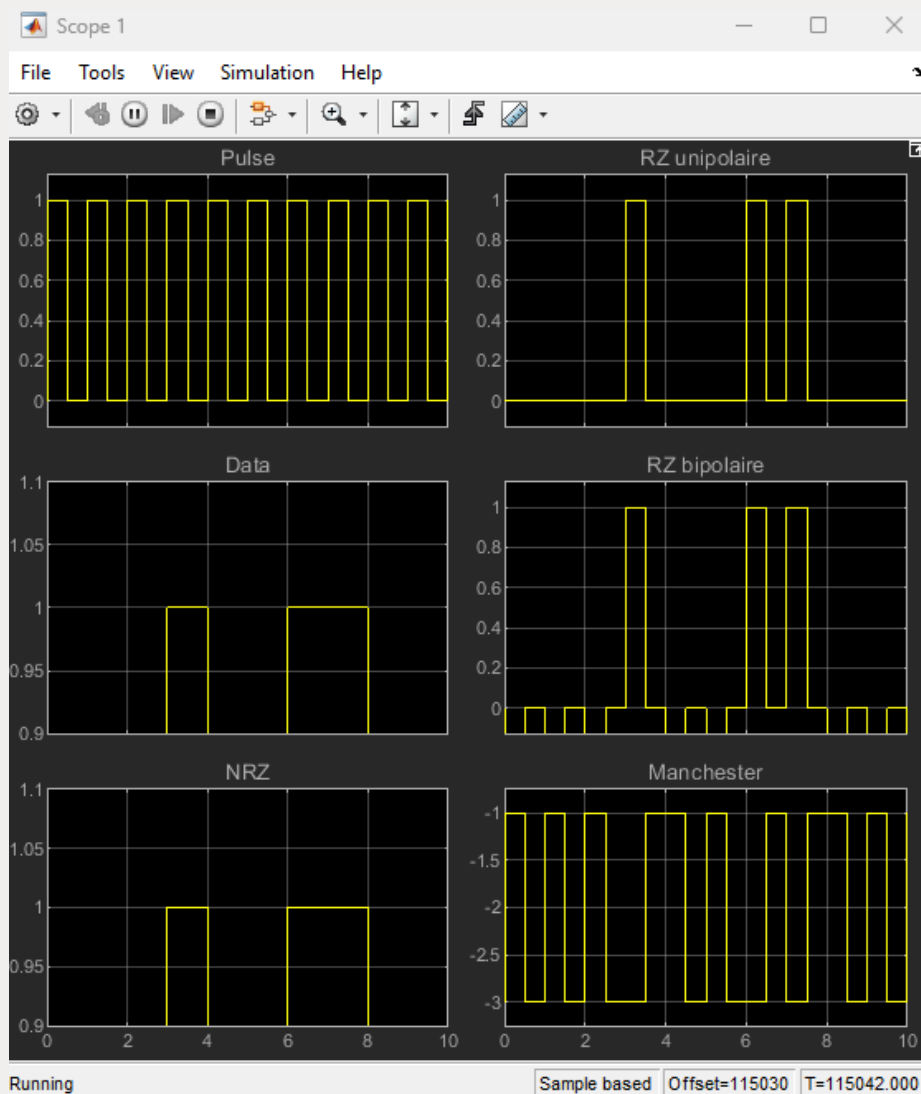
Code	Composante continue (V)	Bande passante (Hz)	Spectre continu	Spectre discret	Valeur Maximale (V)
NRZ (bipolaire)	≈ 0	2	Oui	Non	≈ 0.12
RZ unipolaire	0 (pas de raie à 0 Hz)	4	Oui	Oui (raies à f≈1 Hz, 3 Hz)	≈ 0.225 (à 1 Hz)
RZ bipolaire	≈ 0.05 (petite raie mesurée à 0 Hz)	4	Oui	Oui (raies d'horloge)	≈ 0.10
Manchester	0 (aucune raie à 0 Hz)	4	Oui	Non	≈ 0.09

- Double cliquez sur la boîte **Bernouilly Binary** et fixer la valeur du paramètre **Probability of zero** à 0. Lancer la simulation et observer l'évolution temporelle des codes. Que pouvez-vous conclure sur le code NRZ par rapport aux codes RZ unipolaire, RZ bipolaire et Manchester ?



Tous les bits = 1 → le **code NRZ** reste **constant**, donc **composante continue forte**.
 Les **codes RZ, bipolaire et Manchester** gardent une **activité périodique** → ils restent exploitables.

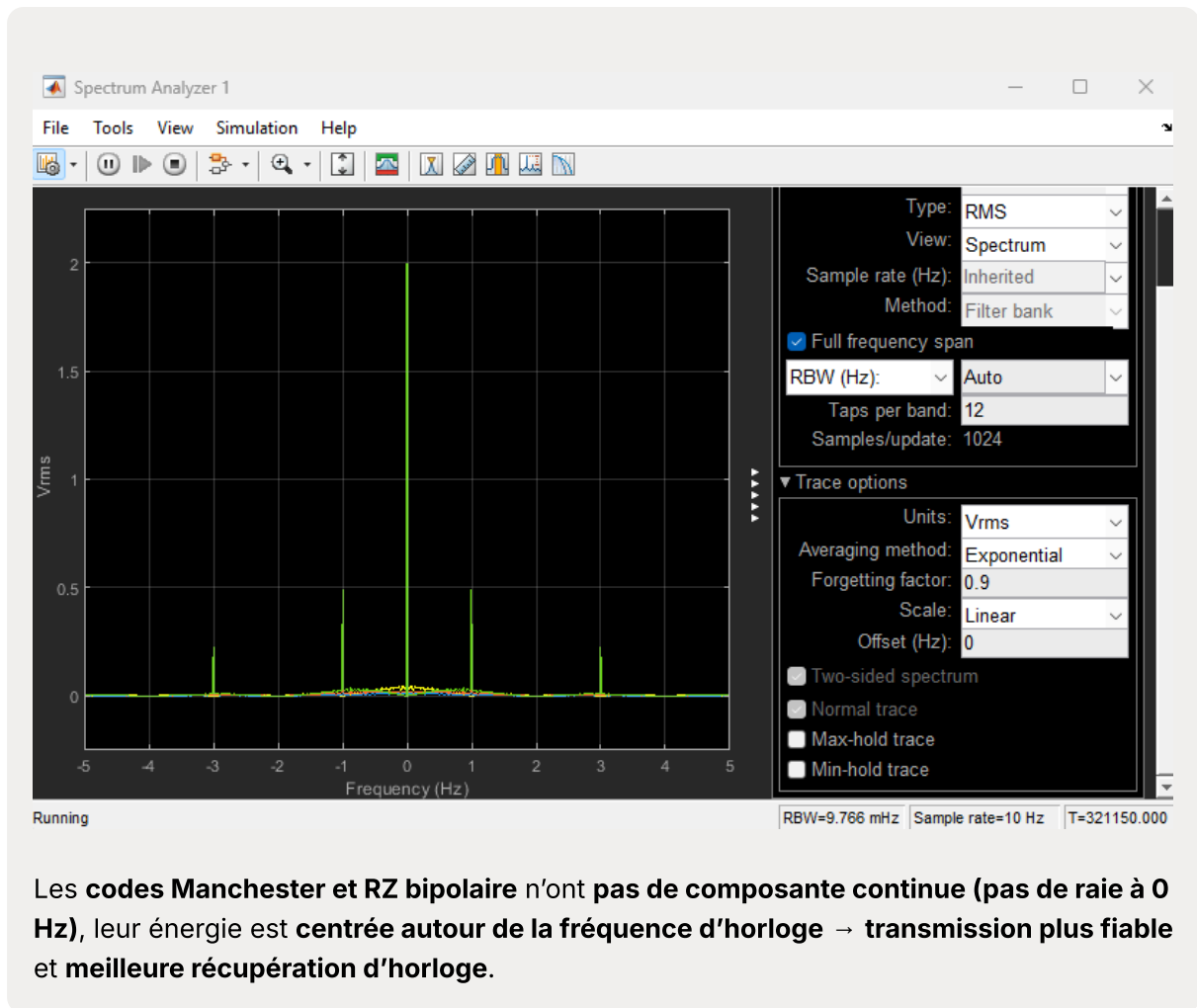
- Fixer la valeur du paramètre **Probability of zero** à 0,95. Lancer la simulation et observer l'évolution temporelle des codes. Que pouvez-vous conclure sur les codes NRZ et RZ unipolaire par rapport aux autres codes RZ bipolaire et Manchester ?



Le **NRZ** et le **RZ unipolaire** deviennent quasi plats (plus de transitions) → **perte de synchronisation**.

Les **codes bipolaire et Manchester** conservent des transitions → **meilleure synchronisation**.

- Fixer la durée de simulation **Stop Time** à la valeur **inf**. Observer les spectres des différents codes. Que pouvez-vous conclure sur les codes Manchester et RZ bipolaire ?



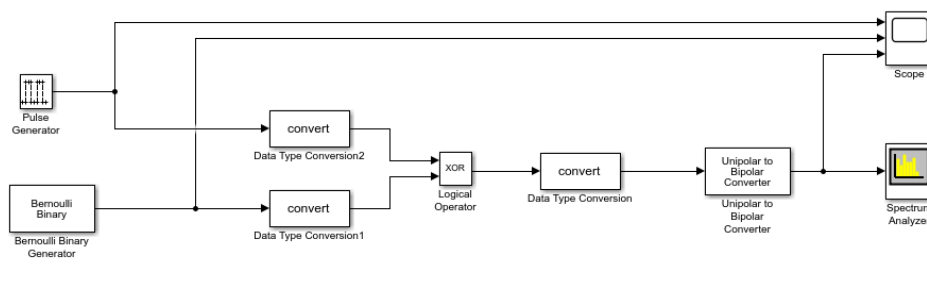
Principe du décodage du code Manchester

On réalise ce schéma :

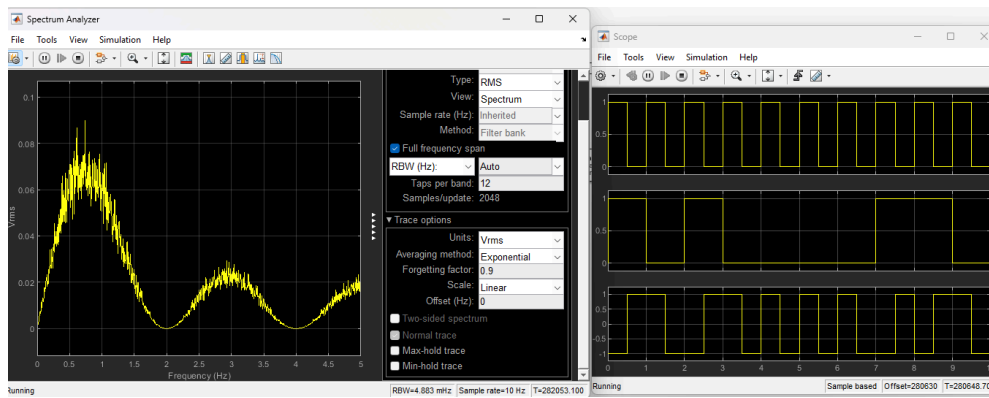
Dans le pulse generator le sample time est a 0.1.

Dans le Bernoulli binary le probability of zero est a 0.5.

Dans le Unipolar to Bipolar converter le M-ary number est a 2.



On obtiens ce résultat :



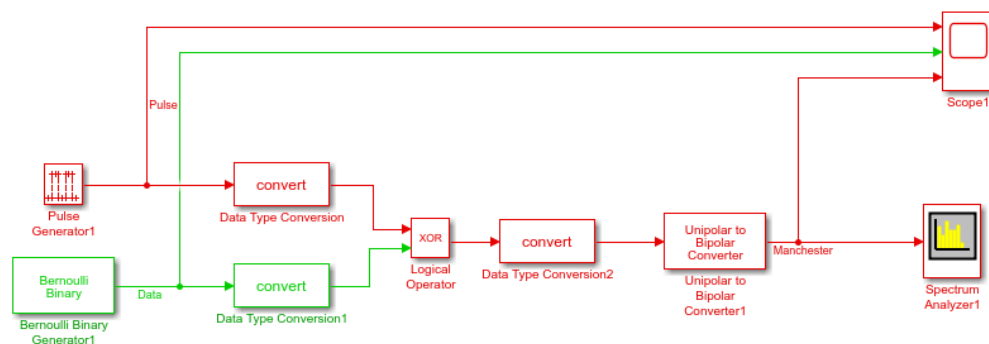
Principe du décodage du code Manchester

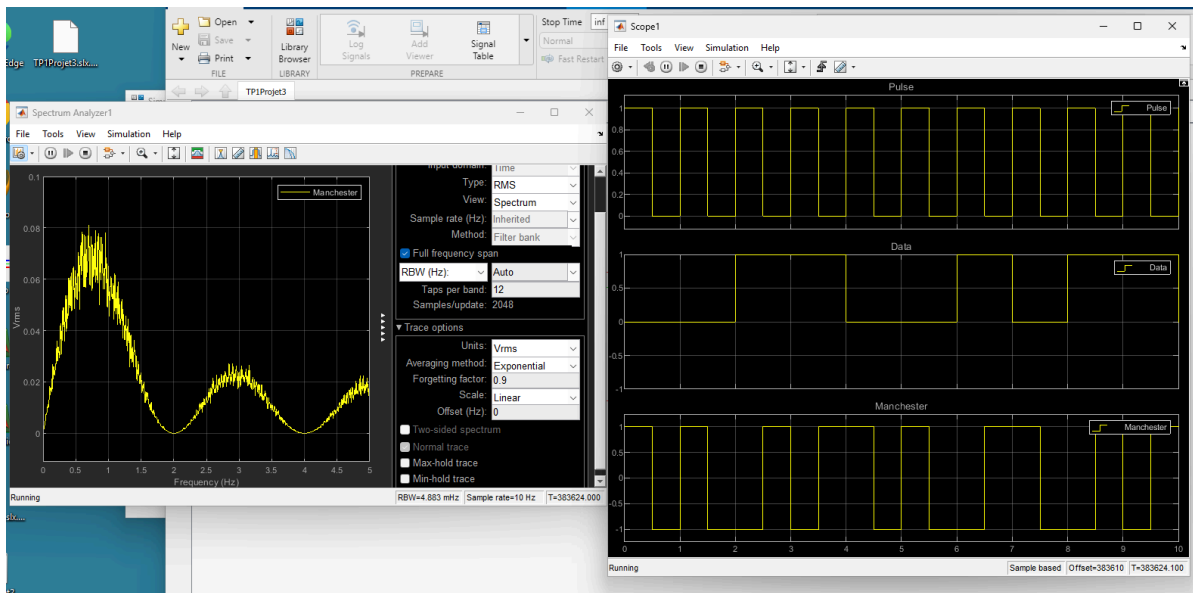
Dans cette partie, nous allons étudier le décodage du code Manchester.

Préparation du fichier de simulation

- Dupliquer le fichier **TP1Projet2** et le renommer **TP1Projet3**.
- Effacer les codes : NRZ, RZ unipolaire et RZ bipolaire.

Votre schéma doit ressembler à celui-ci :

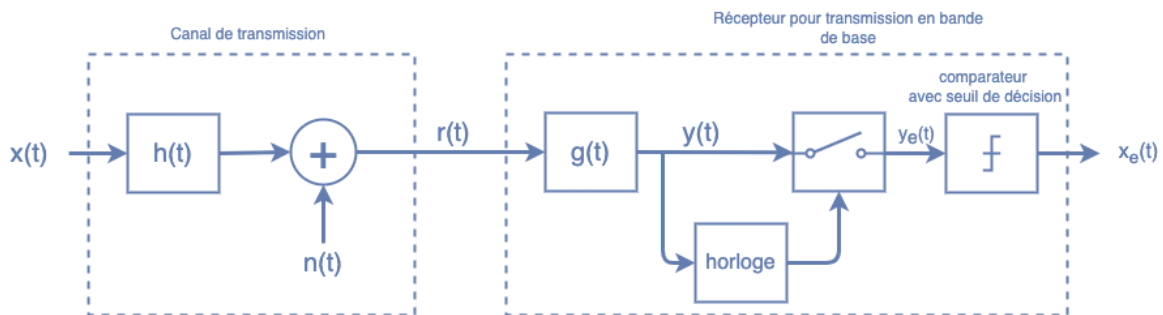




Principe du décodage

Le cours (diapo 55/131) indique que l'estimation du signal Manchester peut être réalisé avec le schéma de principe suivant :

Le signal reçu ($r(t)$) est déformé par le canal de transmission, c'est à dire qu'il ne ressemble pas au signal émis.

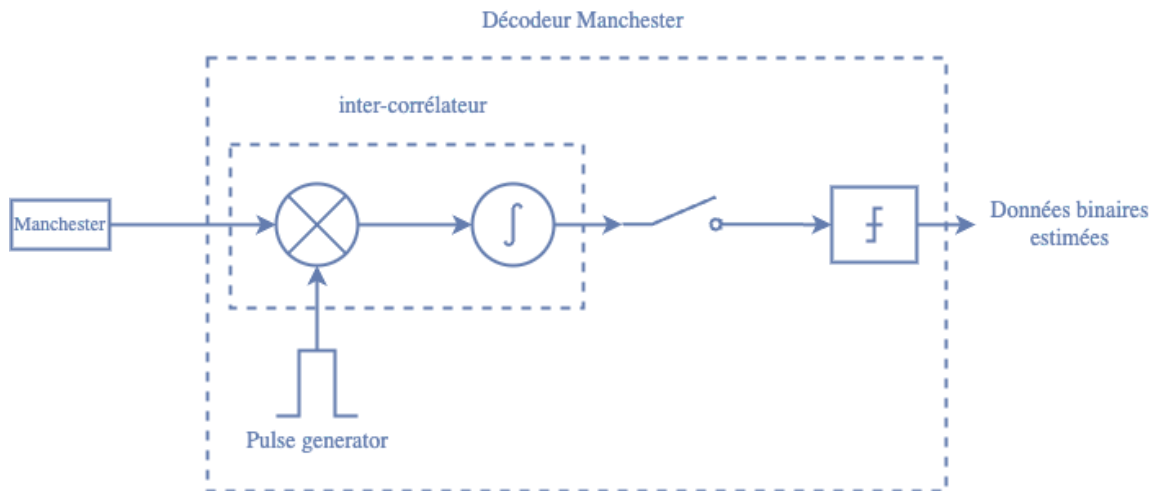


Le schéma présente une méthode pour démoduler le signal :

- le signal reçu $y(t)$ est filtré par un filtre adapté $g(t)$ puis échantillonné à des instants précis et une fonction de décision doit statuer sur la valeur du bit émis.

Une seconde méthode consiste à inter-corréler le signal Manchester avec le **Pulse Generator** et ensuite utiliser une fonction de décision.

Le schéma suivant décrit cette méthode :



Cette méthode est mathématiquement appelée inter-corrélation.

Voici une explication qualitative de l'intercorrélation.

La source est : [Site de Vincent Mazet](#)

Décodeur Manchester

Le schéma sous Simulink est présenté sur la figure ci-dessous

En communications numériques, il n'est pas rare que le récepteur du système de communication reçoive un signal de l'émetteur qui soit très brouillé (on dit qu'il est *bruité*). Par exemple, si le récepteur reçoit le signal x représenté Fig. 6, et que ce signal est en réalité une suite d'échelons d'amplitude -1 (représentant le bit 0) ou $+1$ (représentant le bit 1), alors l'intercorrélation de x avec un échelon y permet de détecter à chaque instant si le signal reçu ressemble à y (dans ce cas, on a reçu un 1) ou pas (on a reçu un 0).

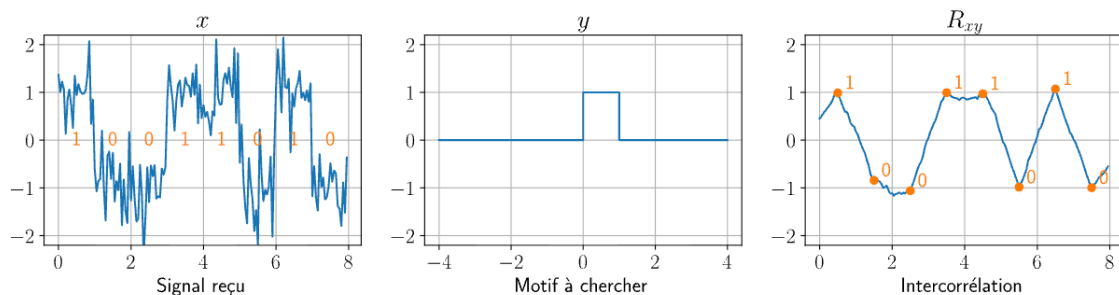
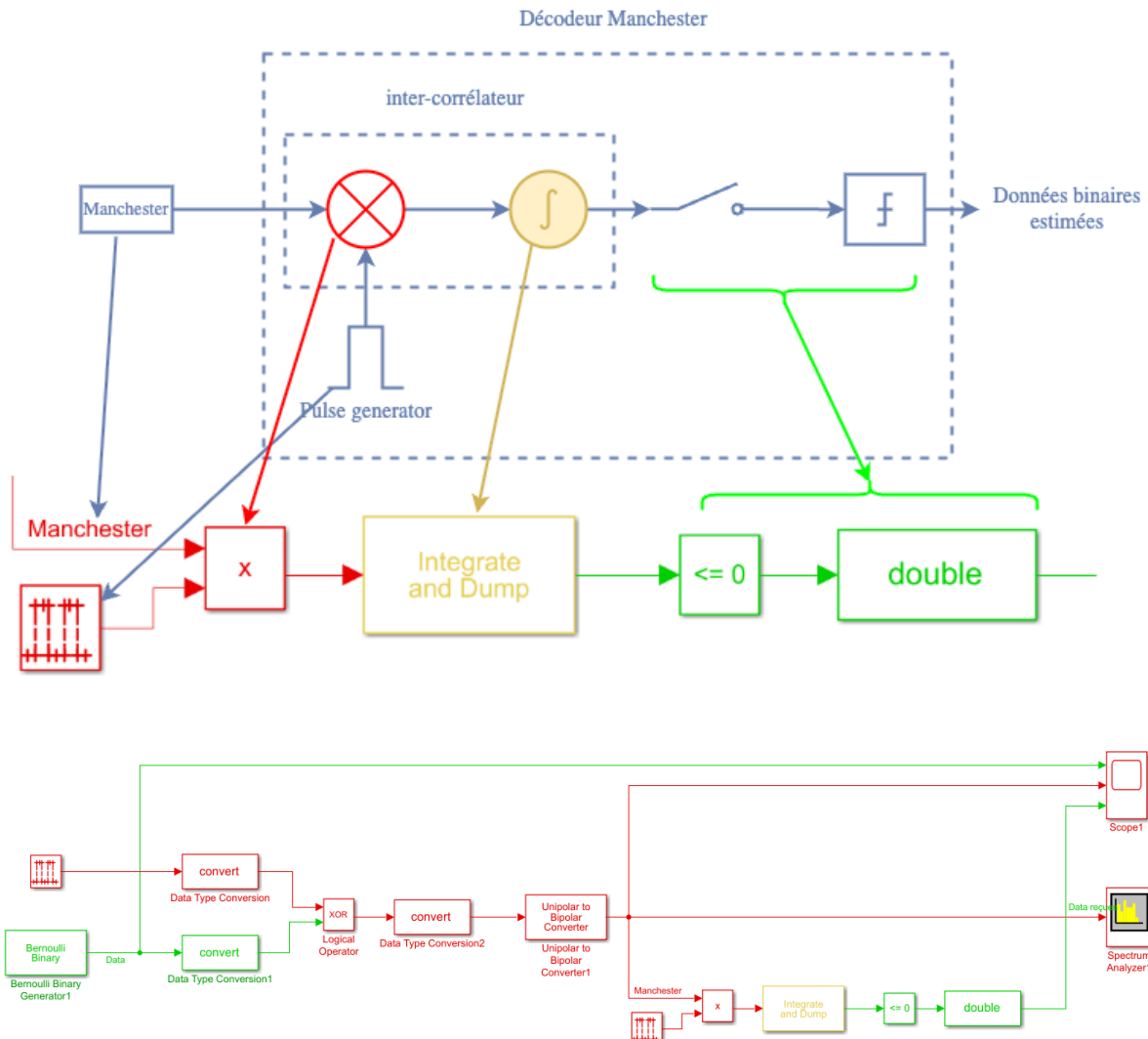
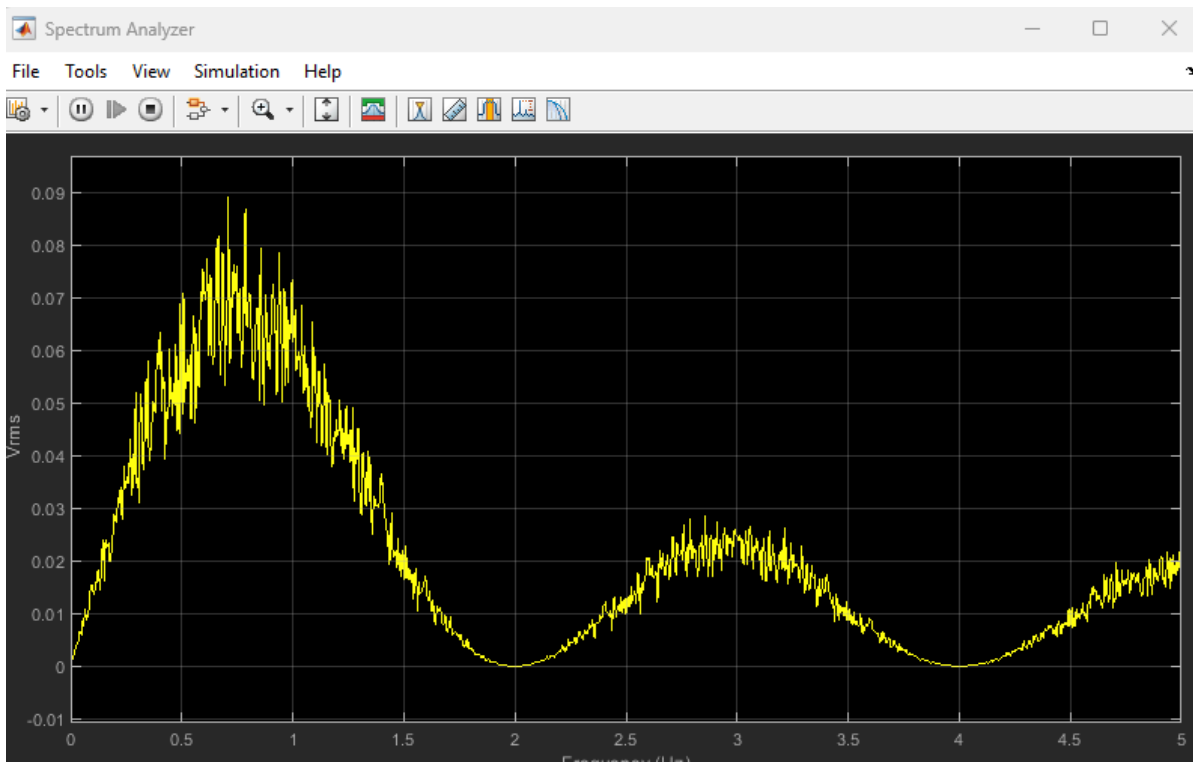


Fig. 6 Intercorrélation de x avec y . Le signal x véhicule le message 10011010 codé en NRZ avec le motif y .



- Copiez le **Pulse Generator** utilisé précédemment. La période est de 1 seconde avec un pas d'échantillonnage de 0,1 s.
- Placer une boîte **Simulink > MATH Operations > Product**
- Placer une boîte **Communications Filters > Comm Filters > Integrate and Dump**. Double cliquez sur la boîte et mettre la valeur du paramètre **Integration period** à 10 (durée de 1 seconde)
- Placer une boîte **Simulink > Logic and Bit Operations > Compare to Zero**. Double cliquez sur la boîte et fixer la valeur du paramètre **Operator** à **<=**.
- Placer une boîte **Simulink > Quick Insert > Signal Attributes > Cast To Double**.



- le **premier bit** peut être tronqué/indéfini (conditions initiales),
- les **niveaux** peuvent être ± 1 au RX alors que l'émission était **0/1** (après conversion unipolaire $\leftrightarrow$ bipolaire), mais les **décisions logiques** sont les mêmes.

1. Quelle remarque pouvez-vous formuler sur les minimas ?

Les **minimas ne tombent plus à zéro** car le **bruit ajoute de l'énergie à toutes les fréquences**, même là où le code Manchester ne contient théoriquement aucune composante. Cela confirme que le bruit **élève le plancher spectral** sans altérer la forme globale du spectre.

Mesure du taux d'erreurs

Dans une transmission numérique, le taux d'erreur bit permet de qualifier une transmission numérique. Il est défini par la relation

$$BER = \frac{N_e}{N}$$

avec:

- N_e : nombre de bits erronés
- N : nombre bits émis.

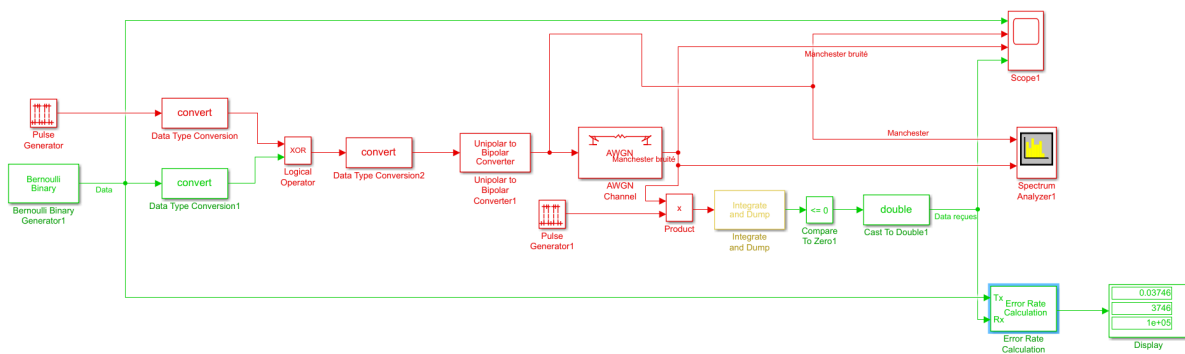
Dans Simulink la boîte **Communications Toolbox > Comm Sink > Error Rate Calculation** permet de mesurer le taux d'erreur.

On associe cette boîte à la boîte **Display** qui permet d'afficher les résultats fournis par la boîte.

Vous avez sans doute remarqué que le signal décodé a un décalage temporel de 1 bit donc de 1 seconde. Pour calculer le taux BER, vous devez retarder le signal original du même temps.

Il faut donc ajuster les paramètres de la boîte **Error Rate Calculation**

- Ajouter la boîte **Communications Toolbox > Comm Sink > Error Rate Calculation**.
- Ajouter la boîte **Simulink > Sinks > Display**
- Faire les réglages pour la boîte **Error rate Calculation**



Dans cette partie vous tracez l'évolution du taux d'erreur en fonction du rapport E_b/N_0 . Pour cela, je vous propose de le faire via la console Matlab en ouvrant l'éditeur intégré à Matlab.

1. Notez le taux d'erreur pour les valeurs de E_b/N_0 suivantes : $E_b/N_0 = [1 \ 2 \ 5 \ 7 \ 9 \ 11 \ 13 \ 14]$
2. À partir de quelle valeur le taux d'erreur est-il nul (pour 100 000 symboles reçus) ?
3. Copier/coller le code suivant dans l'éditeur et sauvegarder votre fichier.

```
%
% Script Matlab
%
EbN0 = [1 2 5 7 9 11 13]; % Valeurs à indiquer pour chaque simulation
BER = []; % Vecteur à compléter par les valeurs relevées sur display
semilogy(EbN0,BER,'bx');
xlabel('Eb/N0 en dB');
ylabel('Taux d'erreur');
grid on;
```

4. Exécuter le script précédent et comparer la courbe à celle du cours (diapo 73/131). Expliquez la différence obtenue.

Ici on viens faire un script et on le complete avec les valeurs obtenu sur le display en faisant varier de [1 2 5 7 9 11 13 14] :

